# Ders 20 Çalışma Özeti

## Ders 20 Çalışma Özeti

### Genel Konular

- Vize ve 1 Mayıs tatilinden sonraki ilk ders. Graflar konusuna giriş yapıldı.
- Graf türleri: null, trivial, yönlü/yönsüz, bağlı, tam, döngülü, DAG, iki parçalı, ağırlıklı graf.
- Grafın saklanması: komşuluk matrisi (adjacency matrix) ve komşuluk listesi (adjacency list).
- Yoğun (dense) ve seyrek (sparse) graf ayrımı, hangi gösterimin ne zaman seçileceği.
- Graf üzerindeki temel işlemler: düğüm/kenar ekleme, silme, arama, gezinme (DFS, BFS).
- Kapsayan ağaç (spanning tree) ve en küçük kapsayan ağaç (minimum spanning tree, MST).
- Açgözlü (greedy) yaklaşım; Prim ve Kruskal algoritmalarının fikri.
- Kruskal algoritması: iki el örneği, `belongs` dizisiyle döngü kontrolü ve C kodu.

### Hocanın Özellikle Vurguladığı Kısımlar

- Bir graf sorusunda önce grafın yönlü mü yönsüz mü olduğuna bakın. Çözüm buna göre değişir.
- Yönsüz grafın komşuluk matrisi simetrik olmak zorundadır. Simetrik değilse ya bilgi hatası vardır ya da graf yönlüdür.
- "Matris mi liste mi daha iyi?" sorusunun mutlak cevabı yok. Karar, kenar sayısına (yoğunluğa) bağlıdır.
  - Seyrek graf: komşuluk listesi.
  - Yoğun graf: komşuluk matrisi.
- Ağaç, grafın özel bir hâlidir. Bir graftan ağaç çıkabilmesi için döngü (cycle) olmamalıdır.
- Kapsayan ağacın iki şartı: bağlılık (connectivity) ve döngüsüzlük (acyclic).
- Kruskal'da kenar ne kadar küçük olursa olsun, döngü oluşturuyorsa alınmaz. Algoritmanın temel kuralı budur.
- Kodda iyileştirme önerisi: eklenen düğümleri işaretleyin. Tüm düğümlere ulaşıldığında döngüyü bitirin, kalan kenarlara bakmayın.

### Kısa Tekrar Notları

- Graf: $G = (V, E)$. $V$ düğümler (vertex/node), $E$ kenarlar (edge).
- Null graf: kenarı olmayan graf. Trivial graf: tek düğümlü graf.
- Bağlı (connected) graf: her düğümden her düğüme yol vardır. Değilse bağlı olmayan (disconnected) graf.
- Tam (complete) graf: her düğüm çifti doğrudan bağlıdır. Ağlardaki karşılığı mesh topolojisidir.
- DAG (Directed Acyclic Graph): döngü içermeyen yönlü graf.
- İki parçalı (bipartite) graf: düğümler iki kümeye ayrılır. Kenarlar yalnız kümeler arasındadır. Örnek: yapay sinir ağında iki ardışık katman.
- Ağırlıklı (weighted) grafta kenarların değeri vardır. Değerler genelde pozitiftir ama negatif de olabilir.
- Matris gösterimi: bağlantı varsa 1 (ya da ağırlık), yoksa 0 (ağırlıklı grafta $\infty$).
- Liste gösterimi: her düğüm için komşularını tutan bir bağlı liste.
- MST: kapsayan ağaçlar içinde toplam kenar ağırlığı en küçük olan. Birden fazla MST olabilir.

- Kruskal: kenarları sırala, en küçükten başla, döngü oluşturmayanı ekle.
- Prim: bir başlangıç düğümü seç, ağaçtan dışarı çıkan en küçük kenarı ekle.

## Detaylı Açıklamalar

### Graf Türleri

Hoca türleri tahtaya numaralandırarak yazdı.

| Tür | Tanım |
|---|---|
| Null graph | Düğüm var, hiç kenar yok. |
| Trivial graph | Tek düğüm (single vertex). |
| Undirected graph | Kenarların yönü yok. |
| Directed graph | Kenarlar oklarla yön belirtir. |
| Connected graph | Her düğüm diğerlerine erişebilir. |
| Complete graph | Her düğüm her düğüme doğrudan bağlı. |
| Cycle graph | Grafın kendisi bir döngüdür. |
| Cyclic graph | Grafın içinde en az bir döngü vardır. |
| DAG | Döngü içermeyen yönlü graf. |
| Bipartite graph | Düğümler iki kümede; kenarlar yalnız kümeler arasında. |
| Weighted graph | Kenarların ağırlığı var (yönlü ya da yönsüz olabilir). |

- Ağırlıklı grafa örnek: haritada şehirler arası mesafe.
- Ağaçlar grafların özel bir türüdür.
  - Döngülü bir graftan bir kenar koparılırsa ağaç elde edilebilir.
  - Kök (root) herhangi bir düğüm seçilebilir. Bu yüzden bir graftan birden fazla ağaç çıkabilir.

### Komşuluk Matrisi ve Komşuluk Listesi

Tahtadaki örnek graf (yönsüz): 0-1, 0-2, 1-2, 2-3.

Komşuluk matrisi:

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 2 | 1 | 1 | 0 | 1 |
| 3 | 0 | 0 | 1 | 0 |

Komşuluk listesi:

```
0 -> 1 -> 2
1 -> 0 -> 2
2 -> 0 -> 1 -> 3
3 -> 2
```

İşlem maliyetleri ($n$ = düğüm sayısı):

| İşlem | Matris | Liste |
|---|---|---|
| Kenar ekleme | $O(1)$ | $O(1)$ |
| Kenar silme | $O(1)$ | Listeyi gezmek gerekir (bkz. ders sonrası notlar) |
| İlklendirme | $O(n^2)$ | $O(n)$ |

- Matrisi ilklendirmek için $n \times n$ hücrenin hepsine bir kez erişmek gerekir. Bu yüzden $O(n^2)$.
- Seyrek bir grafı matriste tutarsanız sıfırlarla dolu $n \times n$ alanı boşa harcarsınız.
- Hocanın ikinci örneği: A–F düğümlü, yalnız birkaç kenarı olan bir graf. Liste gösteriminde kenarlar tek yönde yer alıyordu. Bu yüzden graf yönlüdür.
- Matrisin avantajları: erişim kolay, iki düğüm arasında kenar var mı sorusu $O(1)$.
- Matrisin dezavantajları: çok yer kaplar; yeni düğüm eklemek pahalıdır.
- Listenin avantajları: az yer kaplar; yeni düğüm eklemek kolaydır; bir düğümün komşularını gezmek hızlıdır.

**Grafların Kullanım Alanları ve İşlemler**

- Haritalarda rota bulma, sosyal ağlarda arkadaşlık ilişkileri, bilgisayar ağlarında yönlendirme (routing).
- Temel işlemler: düğüm ekleme, kenar/düğüm silme, arama, gezinme.
- Ağaçlarda inorder/preorder/postorder vardı. Graflarda ise iki ana gezinme yöntemi var:
  - Derinlik öncelikli arama (Depth First Search, DFS).
  - Genişlik öncelikli arama (Breadth First Search, BFS).

**Kapsayan Ağaç (Spanning Tree)**

- Bir grafın kapsayan ağacı, tüm düğümleri içeren, bağlı ve döngüsüz bir alt graftır.
- Bağlı olmayan bir yapı ağaç değildir. Birden fazla ağaçtan oluşan yapıya orman (forest) denir.
- Bir graftan birden fazla kapsayan ağaç çıkarılabilir.

Tahtadaki örnek: a-b (1), a-c (5), a-d (2), c-d (3).

| Ağaç | Kenarlar | Ağırlık |
|---|---|---|
| $T_1$ | a-b, a-d, c-d | $1+2+3=6$ |
| $T_2$ | a-b, a-c, c-d | $1+5+3=9$ |
| $T_3$ | a-b, a-c, a-d | $1+5+2=8$ |

- En küçük toplam $T_1$'de: $w(T_1)=6$. MST budur.
- Kullanım alanı olarak gezgin satıcı problemi (Traveling Salesman Problem) örneği verildi.

**Açgözlü (Greedy) Yaklaşım**

- Her adımda o anki seçenekler içinden en iyisini seçerek ilerlenir.
- Bu seçimler sonunda genel en iyi sonuca ulaştırıyorsa algoritmaya açgözlü algoritma denir.
- Prim ve Kruskal açgözlü algoritmalardır.
- İki yaklaşım:
  - Kruskal: kenarları küçükten büyüğe sırala. Her seferinde listedeki en küçük kenarı al.
  - Prim: bir başlangıç düğümü seç. Bu düğümden (ağaçtan) diğer düğümlere giden en kısa kenarı al.

**Kruskal Algoritması: Birinci Örnek**

Graf: 6 düğüm (a–f), 10 kenar.

| Sıra | Kenar | Ağırlık | Sonuç |
|---|---|---|---|
| 1 | b-c | 1 | Alındı |
| 2 | e-f | 2 | Alındı |
| 3 | a-b | 3 | Alındı |
| 4 | b-f | 4 | Alındı |
| 5 | c-f | 4 | Döngü, alınmadı |

| Sıra | Kenar | Ağırlık | Sonuç |
|---|---|---|---|
| 6 | a-f | 5 | Döngü, alınmadı |
| 7 | d-f | 5 | Alındı |
| 8 | a-e | 6 | Bakılmadı |
| 9 | c-d | 6 | Bakılmadı |
| 10 | d-e | 8 | Bakılmadı |

- Eşit ağırlıklı kenarlarda (b-f, c-f) harf sırası önce olan alındı.
- d-f eklendiğinde tüm düğümlere ulaşıldı. Kalan kenarlara bakmaya gerek kalmadı.
- MST ağırlığı: $1 + 2 + 3 + 4 + 5 = 15$.
- $V = 6$ düğüm için MST'de $V - 1 = 5$ kenar vardır.

### Döngü Kontrolü: `belongs` Dizisi

- Her düğümün ait olduğu kümeyi gösteren bir dizi tutulur.
- Başlangıçta her düğüm kendi kümesindedir: `belongs[i] = i`.
- Yeni kenarın iki ucunun küme değerine bakılır:
  - Değerler aynıysa iki düğüm zaten aynı kümededir. Kenar döngü oluşturur, alınmaz.
  - Değerler farklıysa kenar alınır. İki küme birleştirilir: bir kümenin bütün elemanları diğerinin değerini alır.
- Tahtadaki örnek:
  - A–G düğümleri 1 değerli, H–K düğümleri 2 değerli iki ayrık küme (disjoint subset).
  - U (1) ile V (2) arasındaki kenar alınır, iki küme tek kümede birleşir.
  - E ile G arasındaki kenar alınmaz, çünkü ikisi de 1 değerli; döngü oluşur.

### Kruskal Algoritması: C Kodu

Hoca kodu OnlineGDB üzerinde gösterdi.

```c
#define MAX 30

typedef struct edge {
  int u, v, w;
} edge;

typedef struct edge_list {
  edge data[MAX];
  int n;
} edge_list;

edge_list elist;

int Graph[MAX][MAX], n;
edge_list spanlist;

void kruskalAlgo() {
  int belongs[MAX], i, j, cno1, cno2;
  elist.n = 0;

  /* Matrisin alt üçgeninden kenar listesini çıkar */
  for (i = 1; i < n; i++)
    for (j = 0; j < i; j++) {
      if (Graph[i][j] != 0) {
        elist.data[elist.n].u = i;
        elist.data[elist.n].v = j;
        elist.data[elist.n].w = Graph[i][j];
        elist.n++;
      }
    }
```

```
  sort();                    /* kenarları ağırlığa göre sırala */

  for (i = 0; i < n; i++)
    belongs[i] = i;          /* her düğüm ayrı kümede */

  spanlist.n = 0;

  for (i = 0; i < elist.n; i++) {
    cno1 = find(belongs, elist.data[i].u);
    cno2 = find(belongs, elist.data[i].v);

    if (cno1 != cno2) {      /* farklı kümeler: döngü yok */
      spanlist.data[spanlist.n] = elist.data[i];
      spanlist.n = spanlist.n + 1;
      applyUnion(belongs, cno1, cno2);
    }
  }
}

int find(int belongs[], int vertexno) {
  return (belongs[vertexno]);
}

void applyUnion(int belongs[], int c1, int c2) {
  int i;
  for (i = 0; i < n; i++)
    if (belongs[i] == c2)
      belongs[i] = c1;
}
```

- Graf komşuluk matrisinde tutulur. `main` içinde $n = 6$ ve matris elle doldurulur.
- `elist`: tüm kenarlar. `spanlist`: MST'ye alınan kenarlar.
- `find`: düğümün küme değerini döndürür.
- `applyUnion`: `c2` kümesindeki tüm düğümleri `c1` kümesine taşır.

### Kruskal Algoritması: İkinci Örnek

Öğrencilerin isteği üzerine 9 düğümlü (0–8), 14 kenarlı bir graf çözüldü.

| Kenar | Ağırlık | Sonuç |
|---|---|---|
| 6-7 | 1 | Alındı |
| 5-6 | 2 | Alındı |
| 2-8 | 2 | Alındı |
| 0-1 | 4 | Alındı |
| 2-5 | 4 | Alındı |
| 6-8 | 6 | Döngü, alınmadı |
| 2-3 | 7 | Alındı |
| 7-8 | 7 | Döngü, alınmadı |
| 0-7 | 8 | Alındı |
| 1-2 | 8 | Döngü, alınmadı |
| 3-4 | 9 | Alındı |
| 4-5 | 10 | Bakılmadı |
| 1-7 | 11 | Bakılmadı |
| 3-5 | 14 | Bakılmadı |

- Her adımda `belongs` dizisi tahtada güncellendi.
- 3-4 eklenince tüm düğümler aynı küme değerini aldı. Bu, tüm düğümlere ulaşıldığını gösterir. Ek diziye gerek kalmadan durulabilir.

- Alınan kenar sayısı $9 - 1 = 8$.
- MST ağırlığı: $1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$ (hoca toplamı söylemedi; tablodan hesaplandı).

**Duyurular**

- Vize kâğıtlarındaki kodların okunması zaman alıyor. Sonuçlar yaklaşık bir hafta içinde açıklanacak.
- Sınav sonrası e-postayla dosya gönderenlerin dosyaları kabul edilmiyor. Teknik sorun yaşayanlar UZEM üzerinden başvurmalı.
- Üçüncü ve son ödev ertesi gün verilecek. Süresi bir haftadan kısa olabilir.
- Dönem projesi bir sonraki hafta verilecek.
- Bir sonraki hafta: Prim algoritması, belki Dijkstra (en kısa yol) ve vize sorularının çözümü.

**Ders sonrası notlar**

- Komşuluk listesinde kenar silme $O(1)$ değildir. Altyazıda iki taraf için de $O(1)$ geçiyor, ama hocanın açıklaması listenin gezilmesi gerektiğidir. Doğrusu: $O(\deg(v))$, en kötü durumda $O(V)$.
- "Cycle graph" ile "cyclic graph" ayrımı: cycle graph ($C_n$) tamamen tek bir döngüden oluşan graftır. Cyclic graph ise en az bir döngü içeren herhangi bir graftır. Hocanın tanımı bu anlamdadır.
- "Bir graftan $n$ tane kapsayan ağaç çıkar" ifadesi kesin bir sayı değildir. Tam graf $K_n$ için kapsayan ağaç sayısı $n^{n-2}$'dir (Cayley formülü).
- MST gezgin satıcı problemini doğrudan çözmez. Gezgin satıcı problemi NP-zordur. MST, metrik gezgin satıcı problemi için en fazla 2 kat kötü bir yaklaşık çözüm üretmekte kullanılır.
- Kruskal yönsüz graflarda çalışır. Kodda yalnız alt üçgen ($j < i$) gezildiği için her kenar bir kez listeye girer.
- Hoca tahtada matris için "dense matris" dedi; kastedilen yoğun graftır.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.